



Document Last Modified: March 7th, 2023

CCC Scrub API Quick Start Guide

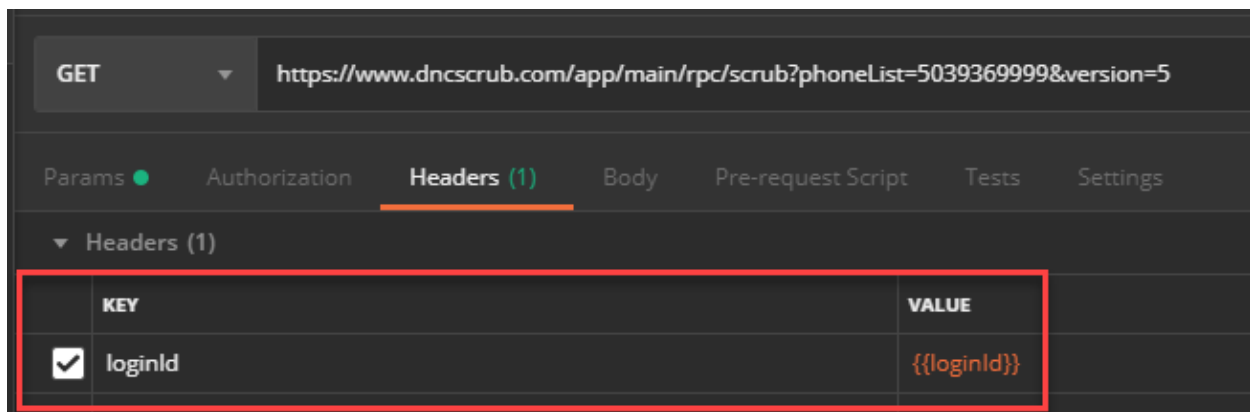
Note that all API calls must be made with TLS 1.2 or above

1 API Concepts

The CCC API is a REST based API that can return either JSON or CSV. All requests to the API must be made via HTTPS using TLS 1.2 or above. If you have questions during your implementation please contact support@dnc.com. We can also provide you a Postman collection to aid in your development

1.1 Authentication

Authentication is done to the API using an API Key we also call your LoginId. The API key is added to the header of each HTTP request. The key of the header row should be **loginid** and the value the API Key generated from the DNCS scrub portal.



1.2 API Operations are Atomic

An important concept to understand is that by default all requests of an API operation either fully complete or no portions of the request completes. For example if you attempt to add multiple phone numbers to your IDNC database and one of your phone number values is not 10 digits, for example you pass a value of "AAA" as a phone number, the entire request will not complete. It is important to pass only clean, 10 digit phone numbers, to the API and take action on any exceptional conditions returned from the API.

1.3 Optional Project Id and Campaign Id parameters

Your DNCS scrub account is ordered into projects and campaigns. If you wish to perform a specific action in a project you can specify the projId parameter. If you wish to specify an action in a specific campaign

you specify the campaignId parameter. When neither parameters are specific the action occurs in the default project and campaign

2 Get an API Key (also called a LoginId)

1. Create a user in User Admin that will be used for the API. Give the user a distinct full name so you will not accidentally delete it such as “API User – Do Not Delete.” The recommended username is “apiuser” though the username does not matter. Make the user an Administrator
2. After the user is created, click “Get API Key.”

The screenshot shows the 'User Admin' interface. At the top, there's a table with columns: All, Username, Full Name, Role, Division; Call Center; Department, Shift, Account, and Last Login Date. Below the table, there's a form for creating or editing a user. The form includes fields for Active (checked), QuickScrub (unchecked), Account (DEMOR), Username (apiuser), Password, Full Name (API User - Do Not Delete), Email, Phone, Extension, Employee Number, Role (Administrator), Division, Call Center Name, Department, Shift (1), Station ID, and Last Login Date (9/13/18 11:09:21 AM). On the right side of the form, there are buttons: Save, Delete, Cancel, Get API Key (highlighted with a red arrow), Show Intro, and Skip Intro. At the bottom left, there's a green arrow pointing to the 'API Key' field, which contains the value: 010EDD083E90CFBFA3193E12612AFF4F83A13F78D140.

3. After you click “Get API Key” an API Key is displayed in the “API Key” area. Make a note of the API Key as this needed in all calls to our API. Even though the API Key text is greyed out it can still be copy-and-pasted.

3 Scrub a Phone Number via the API

1. Develop software that sends an HTTP GET or HTTP POST to the following URL:
2. [https://www.dncscrub.com/app/main/rpc/scrub?phoneList=\\$PHONENUMBER&version=\\$APIOUTPUTVERSION&output=\\$OUTPUT](https://www.dncscrub.com/app/main/rpc/scrub?phoneList=$PHONENUMBER&version=$APIOUTPUTVERSION&output=$OUTPUT)
3. For authentication the HTTP Header must include an HTTP Header row with the key **loginId** and the value retrieved from the User Admin screen.
4. Parameters
 - a. **\$PHONENUMBER** should be the phone number you wish to scrub.
 - b. **\$APIOUTPUTVERSION** should be 5
 - c. **\$OUTPUT** is an optional parameter that can be either csv or json. If not specified CSV output is return
5. Example API call to scrub the phone number 7075276405

```
curl --location --request GET
'https://www.dncscrub.com/app/main/rpc/scrub?phoneList=7075
276405&version=5&output=json' \
--header 'loginId: xxxx'
```

Returns

```
[
  {
    "Phone": "7075276405",
    "ResultCode": "D",
    "Reserved": "",
    "Reason": "National (USA) 2003-06-01;;;",
    "RegionAbbrev": "CA",
    "Country": "US",
    "Locale": "Santa Rosa",
    "CarrierInfo": "9740;RBOC;\\"AT&T California:AT&T California\\\"",
    "NewReassignedAreaCode": "",
    "TZCode": "4",
    "CallingWindow": "",
    "UTCOffset": "-420",
    "DoNotCallToday": "",
    "CallingTimeRestrictions": "4",
    "EBRTType": "",
    "IsWirelessOrVoIP": "0",
    "LineType": "AllOther"
  }
]
```

6. To understand the output, please see the **Output Guide** document to understand all the fields.

4 Scrub Multiple Phone Numbers

Multiple phone numbers can be submitted by concatenating together phone numbers in the phoneList parameter by commas. Example phoneList=5039367187,7079367187 would scrub both 5039367187 and 7079367187. **Important: If you will scrub more than 50 phone numbers, you must use HTTP POST instead of HTTP GET. In addition for more than 50 phone numbers we recommend output=CSV.**

When multiple phone numbers are scrubbed and JSON is specified as the output, the output looks like this:

```
[
  {
    "Phone": "5039367187",
    "ResultCode": "D",
    "Reserved": "",
    "Reason": "Litigator",
    "RegionAbbrev": "OR",
    "Country": "US",
    "Locale": "Portland",
    "CarrierInfo": "5820;WIRELESS;\\"Verizon Wireless:Verizon Wireless\\\"",
    "NewReassignedAreaCode": "",
    "TZCode": "4",
    "CallingWindow": "",
    "UTCOffset": "-420",
    "DoNotCallToday": ""
  }
]
```

```
    "CallingTimeRestrictions": "4",
    "EBRTType": "",
    "IsWirelessOrVoIP": "1",
    "LineType": "Wireless"
  },
  {
    "Phone": "7075276405",
    "ResultCode": "D",
    "Reserved": "",
    "Reason": "National (USA) 2003-06-01;;;",
    "RegionAbbrev": "CA",
    "Country": "US",
    "Locale": "Santa Rosa",
    "CarrierInfo": "9740;RBOC;\AT&T California:AT&T California\"",
    "NewReassignedAreaCode": "",
    "TZCode": "4",
    "CallingWindow": "",
    "UTCOffset": "-420",
    "DoNotCallToday": "",
    "CallingTimeRestrictions": "4",
    "EBRTType": "",
    "IsWirelessOrVoIP": "0",
    "LineType": "AllOther"
  }
]
```

5 Add to or Delete from Internal DNC List

URL:

[https://www.dncscrub.com/app/main/rpc/pdnc?phoneList=\\$PHONENUMBER&actionType=\\$ACTIONTYPE&projId=OptionalProjID](https://www.dncscrub.com/app/main/rpc/pdnc?phoneList=$PHONENUMBER&actionType=$ACTIONTYPE&projId=OptionalProjID)

1. Parameters

- a. **\$PHONENUMBER** should be the phone number you wish to add or remove.
- b. **\$ACTIONTYPE** should be the following:
 - i. add - Adds a phone number to Internal DNC
 - ii. remove - Removes a phone number to Internal DNC

In addition **actionType** can be count or status. See full Enterprise Edition document for documentation on this.

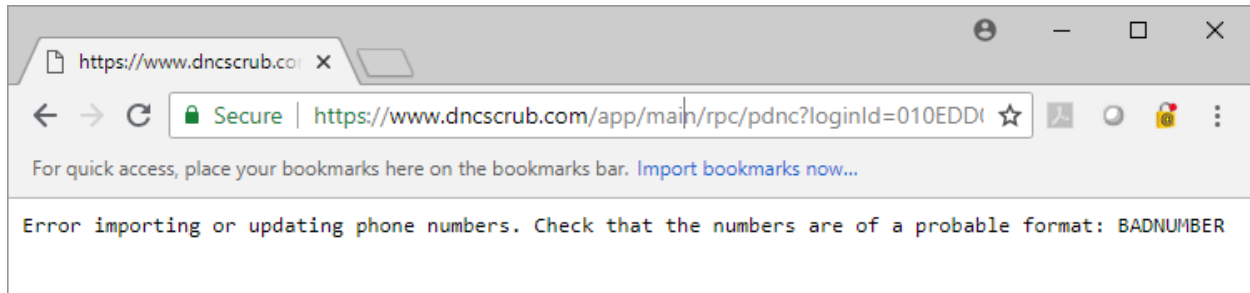
- c. **\$projId** an optional project id to add the IDNC number too.

Example:

```
curl --location --request GET
'https://www.dncscrub.com/app/main/rpc/pdnc?phoneList=5039367187&
actionType=add' --header 'loginId: xxxxxxxx'
```

If the request is successful an HTTP response will be returned with a status code 200 and no body. If unsuccessful a response will be returned with a status code of 4xx and the body containing the reason. Example:

<https://www.dncscrub.com/app/main/rpc/pdnc?phoneList=BADNUMBER&actiontype=add>



Remove Example:

<https://www.dncscrub.com/app/main/rpc/pdnc?phoneList=5039367187&actiontype=remove>

Like Add, if the request is successful an HTTP response will be returned with a status code 200 and no body. If unsuccessful a response will be returned with a status code of 4xx and the body containing the reason. Example:

6 Add to EBR List

URL: <https://www.dncscrub.com/app/main/rpc/ebr>

6.1 Add to EBR

Example:

<https://www.dncscrub.com/app/main/rpc/ebr?>

[1!ColValue.phoneNumber=7075712071&1!ColValue.dateOfLastContact=06/14/11&1!ColValue.type=S&2!ColValue.phoneNumber=7072842774&2!ColValue.dateOfLastContact=07/02/11&2!ColValue.type=I](https://www.dncscrub.com/app/main/rpc/ebr?1!ColValue.phoneNumber=7075712071&1!ColValue.dateOfLastContact=06/14/11&1!ColValue.type=S&2!ColValue.phoneNumber=7072842774&2!ColValue.dateOfLastContact=07/02/11&2!ColValue.type=I)

2. Parameters

- a. *x!ColValue.phoneNumber* (required – 10 digit phone number to be added)
- b. *x!ColValue.dateOfLastContact* (required – date of last contact; should be in the format of MM/DD/YY, formats MM/DD/YYYY and YYYY-MM-DD are also accepted)
- c. *x!ColValue.type* (required – type of EBR; should be either: S for Sale/Purchase, I for Inquiry, P for Permission, or T for Trial (currently only used for Newspaper Trials in North Dakota)) (not case-sensitive)
- d. *x!ColValue.dateObligationEnds* (optional – date when written obligation on your or your customer's part ends – used in New Jersey only)
- e. *x!ColValue.referenceNum* (optional – your internal string you could use to track this EBR)
- f. *x!ColValue.brand* (optional – your internal string meant to store the company/product/brand name that was used to establish the EBR with the customer – this data is mostly for future use)
- g. *x!ColValue.email* (optional – 100 characters or less)

Example:

<https://www.dncscrub.com/app/main/rpc/ebr?1!ColValue.phoneNumber=7075712071&1!ColValue.dateOfLastContact=06/14/11&1!ColValue.type=S&2!ColValue.phoneNumber=7072842774&2!ColValue.dateOfLastContact=07/02/11&2!ColValue.type=I>

Like the Internal DNC APIs, if the request is successful an HTTP response will be returned with a status code 200 and no body. If unsuccessful a response will be returned with a status code of 4xx and the body containing the reason.

6.2 Advanced Usage of EBR API

In addition to adding records, you can also do Scrub+EBR which will first add the phone number to your EBR database and then scrub it informing you if you have an exemption to the DNC rules for that phone number.

There is also an optional parameter keepBetterEBR. If it is 1, if an EBR records already exists for the phone number being added and the existing EBR is "better" than the EBR being added, the existing EBR will not be overridden. Federal EBR expiration dates are used to determine the best EBR.

7 Additional APIs

If you need to perform more advanced operations we have the following APIs our support team can help you with

- **Higher Performance Litigator API** – Used for scenarios where only litigator checking is needed and the highest performance is needed
- **Reassigned ID aka Identity Verification API** – Identifies if a phone number has been reassigned. The main use case of this API is to identify if a phone number
- **Residential or Business Phone Number Identification API** – Identifies if a phone number is residential or business
- **Carrier Identification API** – Identifies in real-time the carrier of the phone numbers. Has the carrier of the phone number even if the phone number was ported.

In addition our support team can provide you with an API Best Practices document.